AI-Based Self-Learning System Behavior Anomaly Agent — Design & Research Plan

1. Purpose of This Document

- Consolidate the **final system design** (architecture, tech choices, scalability).
- Define **implementation phases** and **areas of research/learning** (ML, systems, infra).
- Serve as a **living blueprint** you can refine as you build.

2. System Overview

Objective:
Build a cross-platform intelligent monitoring system that:

- Learns *normal* behavior of machines/environments.
- Detects anomalies using **unsupervised ML** (no labels, no fixed thresholds).
- Handles restarts, data gaps, heterogeneous environments (Windows, Linux, WSL, containers).
- Scales from a **single laptop** to **many thousands of servers**.
- Provides **explainable anomaly insights**, not just scores.

3. Final Architecture (Polyglot, Scalable, Explainable)

3.1 High-Level Architecture

```
┌──────────────────────────────────────────────────┐
│              Agent Layer (Rust)                  │
│  [Collector] [Env Detector] [State Manager]      │
└──────────────────┬───────────────────────────────┘
                   │ gRPC / Protobuf
┌──────────────────▼───────────────────────────────┐
│       Ingestion Layer (Go, stateless)            │
│   Validation | Dedup | Gap Marking | Publish     │
└──────────────────┬───────────────────────────────┘
                   │
                   ▼
            [Message Queue]
      (Redis Streams / NATS / Kafka)
                   │
```

```
        ┌──────────┴──────────┐
        ▼                     ▼
[Feature Workers] (Go/Python)   [Trainer] (Python)
        │                       │
        ▼                       │
[Inference Workers] (Python + SHAP)│
        │                       │
        ▼                       │
+──────────────────────────────────────────+
| TimescaleDB (metrics), Postgres (meta/models) |
| Redis (cache, warm-up state, leader election) |
+──────────────────────────────────────────+
                  │
          ┌──────┴──────┐
          ▼             ▼
[Baseline Intelligence] [Correlation Engine]
          │             │
          └──────┬──────┘
                 ▼
          [Alerting + Feedback]
                 │
                 ▼
            Dashboard / REST API
```

4. Components (Final Version)

4.1 Agent Layer (Rust)

Role: Efficient, low-overhead data collection on each machine.

Key Responsibilities:

- **System Metrics:** CPU, memory, disk, load, I/O.
- **Process Metrics:** top N processes by CPU/mem, process names, PID, lifetime.
- **Network Metrics:** bytes in/out, connection counts, (optionally) remote endpoints.
- **Environment Detection:**
  - OS type, version.
  - WSL vs native Linux.
  - Container vs host.
  - Virtualization depth (bare metal → VM → container).
- **Session & Warm-up:**
  - Maintain a **session token** in a local state file.
  - On change: classify restart:
    - Level 1 – soft restart (same boot).
    - Level 2 – full reboot.

        - Level 3 – environment change (e.g., moved into container).
- **Gap Detection:**
    - Timestamp of last successful send.
    - Classify gaps: `none`, `micro_gap`, `short_gap`, `long_gap`.

Why Rust:

- Single static binary → easy deployment.
- Very low CPU/memory overhead.
- Safe long-running background service (no GC pauses).

4.2 Ingestion Layer (Go)

Role: Entry point for all agent data; designed for **massive concurrency**.

Responsibilities:

- Expose gRPC (and/or REST) endpoints.
- Validate protobuf payloads, ensure required fields.
- Implement **idempotency** (hash of `agent_id + timestamp`) to avoid duplicates.
- Mark data gaps using timestamps from agent and server.
- Publish validated messages to message queue.

Scalability:

- Stateless Go services behind Envoy/NGINX.
- Horizontal scaling via Kubernetes HPA.
- Handles thousands of simultaneous agents via goroutines.

4.3 Message Queue

Role: Decouple ingestion, feature computation, and ML inference.

Choice (by scale):

- < 500 agents → Redis Streams.
- 500–5,000 → NATS JetStream.
- 5,000 → Kafka.

Usage:

- Topic/stream per logical pipeline (e.g., `metrics_raw`, `features_ready`, `anomalies`).
- Backpressure handling via consumer lag metrics.
- Allows separate scaling of ingestion and ML.

4.4 Storage Layer

Databases:

1. **TimescaleDB (PostgreSQL extension)**
   - For time-series metrics and features.
   - Hypertables for metrics (chunk interval ~1 hour).
   - Compression after 7 days.
   - Hot/warm/cold tiering:
     - 0–7 days: full resolution, uncompressed.
     - 7–30 days: compressed, 1-min aggregates.
     - 30+ days: S3/object storage with hourly aggregates.
2. **PostgreSQL (metadata & models)**
   - Agent registry, environment profiles.
   - Model metadata (version, training window, performance metrics).
   - Feedback labels, alert history.
3. **Redis (cache/state):**
   - Warm-up status per agent.
   - Leader election for alert manager.
   - Temporary correlation buffers.

4.5 Feature Engineering Layer

Input: Raw snapshots from queue + historical context from TimescaleDB.

Features:

- **Statistical:**
  - Rolling mean/variance for CPU, memory, disk usage.
  - Exponential moving averages.
- **Temporal:**
  - Hour of day, day of week, weekend/weekday flag.
  - Time since last activity.
- **Process-level:**
  - Frequency of each process.
  - Resource usage trends per process.
  - **Novelty score** (first seen vs known).

- **Network:**
  - Δ bytes in/out, Δ connection counts.
  - Rate of new remote endpoints.
- **Entropy-based:**
  - Entropy of process name distribution in window.
  - Change in entropy (Δentropy) vs baseline.
- **Drift signals:**
  - Outputs from online drift detectors (e.g., ADWIN).
- **Gap & env context:**
  - Gap type feature.
  - Environment type (host/WSL/container) as categorical.

Implementation can be **Go or Python**; Python easier initially (NumPy/Pandas), Go for performance later.

4.6 AI Model Engine

Models:

- **Isolation Forest** (primary, scikit-learn).
- **One-Class SVM** (for boundary refinement).
- **Optional Autoencoder** (for temporal sequences if needed).

Training:

- Unsupervised; uses past N days (e.g., last 7–30 days).
- Sliding window retraining.
- Exclude known unstable periods (deploys, heavy maintenance).

Inference:

- Real-time scoring from feature vectors.
- Ensemble scoring: weighted combination of model scores.

Explainability:

- Use **TreeSHAP** for Isolation Forest.
- Compute feature contributions per anomaly.
- Generate human-readable reasons from top contributors, e.g.:

> "High sustained CPU (rolling mean +0.31) plus new process `db_backup_x` (novelty +0.29) different from typical overnight pattern."

4.7 Baseline Intelligence & Drift Handling

Goal: Separate *drift* (new normal) from *anomaly* (unexpected event).

- Use **ADWIN** (or similar) for online drift detection on key features.
- If drift is detected:
  - Label context as `drift_context = "changing"`.
  - Reduce alert sensitivity or suppress until new baseline stabilizes.
- **Drift vs Anomaly Rule Layer:**
  - Drift high + anomaly low → treat as *drift*, update baseline.
  - Drift low + anomaly high → *true anomaly*.
  - Both high → anomaly with drift context.

4.8 Cross-Agent Correlation Engine

Role: Correlate anomalies across multiple environments:

- Example patterns:
  - WSL CPU spike + host network spike → cross-boundary anomaly.
  - Container new process + host memory spike → possible lateral movement.

Implementation:

- Go service consuming from queue, grouping by **time windows** (e.g., 30s).
- Correlation rules in configuration (pluggable).
- Outputs **correlation score** added to anomaly events.

4.9 Alerting & Feedback

Alerting:

- Channels:
  - CLI logs, REST endpoint, webhooks, email (optional).
- **Fatigue control:**
  - Deduplicate similar alerts within a time window.
  - Escalation tiers based on anomaly score and correlation score.
- Respect:
  - Warm-up state.
  - Gap type (reduce confidence after long gaps).

Feedback:

- User can label anomalies:

    - `false_positive`
    - `true_anomaly`
    - `expected_change` (e.g., "I just installed new software")
- Feedback fed into:
    - Model reweighting (change ensemble weights).
    - Controlled baseline updates for `expected_change`.

4.10 Dashboard / API

- REST (FastAPI) and optional web UI.
- Functions:
    - View anomalies, filter by environment, time range, severity.
    - See SHAP-based explanations per anomaly.
    - View metrics time-series.
    - Submit feedback labels.
- All reads go to **DB read replicas**.

5. Backend Scalability Strategy

5.1 Horizontal & Vertical Scaling

- **Agents (Rust):** 1 per device, minimal resource use.
- **Ingestion (Go):** Horizontally scaled; stateless; auto-scale by CPU & queue lag.
- **ML (Python):** Worker pattern; separate pools for features, inference, training.
- **DBs:**
    - TimescaleDB with read replicas, compression, partitioning.
    - Redis Cluster for high availability.

5.2 Deployment Modes

- **Local Mode:**
    - All components on one machine.
    - Redis Streams + PostgreSQL.
- **Centralized Mode:**
    - Kubernetes, message queue, separate DB cluster.
- **Hybrid:**
    - Local inference in Python on each machine.
    - Periodic export of summaries to central cluster.

6. Project Structure (Final)

```
project/
  agent/             # Rust
    src/
      collector.rs
      environment.rs
      state_manager.rs
      gap_detector.rs
    Cargo.toml

  ingestion/         # Go
    main.go
    routes.go
    schemas.go
    dedup.go

  correlation/       # Go
    engine.go
    rules.go

  alerts/            # Go
    alert_manager.go
    escalation.go
    dedup_filter.go

  ai/                # Python
    features.py
    model.py
    trainer.py
    inference.py
    drift_detector.py
    explainer.py

  baseline/          # Python
    baseline_manager.py
    drift_classifier.py

  feedback/          # Python
    handler.py

  dashboard/         # Python (FastAPI)
    main.py
    routes.py

  storage/
    db.py            # DB connection management
    models.py        # ORMs/schemas
    cache.py         # Redis helpers

  proto/
    snapshot.proto
    alert.proto
```

```
  config/
    settings.toml
```

7. Research & Learning Plan

Break your learning into **tracks**. You don't need to master everything at once; follow a phased plan.

7.1 Track A – Systems & Rust (Agent)

Goals:

- Write a safe, efficient background service.
- Collect system metrics on Linux/Windows/WSL/container.

Topics:

- Rust basics (ownership, lifetimes, async).
- OS APIs:
  - Linux: `/proc`, `/sys`, loadavg, stat.
  - Windows: WMI / Performance Counters.
- Detecting containers (cgroup info, environment clues).
- Writing systemd services & Windows services.
- gRPC client implementations in Rust.

Action Plan:

1. Build a small Rust CLI that prints CPU/memory every second.
2. Extend to environment detection (host vs container vs WSL).
3. Add gRPC client to send metrics to a test Go server.

7.2 Track B – Go Backend & Scalability

Goals:

- Build a high-throughput, low-latency ingestion API.
- Understand message queues and concurrency.

Topics:

- Go basics (goroutines, channels, contexts).
- gRPC server in Go (with Envoy or direct).
- Redis Streams / NATS / Kafka basics.
- Patterns: worker pools, backpressure, graceful shutdown.

- PgBouncer, connection pooling to Postgres/TimescaleDB.

Action Plan:

1. Build a simple Go gRPC server that accepts dummy snapshots.
2. Integrate Redis Streams or NATS.
3. Build a small worker that reads from queue and logs messages.
4. Add load testing (e.g., `vegeta`, `k6`) to measure throughput.

7.3 Track C – Time-Series DB & Data Modeling

Goals:

- Model and store metrics efficiently at scale.
- Learn to query and aggregate time-series data.

Topics:

- TimescaleDB basics: hypertables, chunks, indexes.
- Compression, retention policies.
- Designing schemas for:
  - system_metrics(agent_id, ts, cpu, mem, ...)
  - process_metrics(agent_id, ts, process_name, cpu, mem, ...)
- Aggregation queries (rollups, downsampling).

Action Plan:

1. Install TimescaleDB locally.
2. Create a hypertable and ingest synthetic metrics.
3. Write queries for:
   - last 1 hour metrics.
   - 1-min averages for last day.
4. Enable compression and test query performance before/after.

7.4 Track D – Machine Learning & Drift

Goals:

- Implement unsupervised anomaly detection for time-series.
- Provide explainability and handle concept drift.

Topics:

- Isolation Forest, One-Class SVM, Autoencoders.

- Feature engineering for time-series and system metrics.
- Drift detection algorithms (ADWIN, DDM).
- SHAP (TreeSHAP) for tree-based models.
- Model evaluation for anomaly detection:
  - Precision/recall on synthetic labeled anomalies.
  - False positive/negative trade-offs.

Action Plan:

1. Offline: Use a Jupyter notebook with synthetic data:
   - Generate normal CPU/mem patterns + injected anomalies.
   - Train Isolation Forest, evaluate.
   - Add SHAP explanation.
2. Add ADWIN to distinguish drift from anomalies.
3. Wrap models into `trainer.py` and `inference.py` with clear interfaces.

7.5 Track E – Observability & Operations

Goals:

- Ensure the system itself is observable and reliable.

Topics:

- Prometheus metrics in Go & Python.
- Grafana dashboards for:
  - ingestion rate
  - queue depth
  - anomaly rate
  - DB performance
- Logging standards (structured logs, correlation IDs).
- OpenTelemetry basic tracing.

Action Plan:

1. Add `/metrics` endpoint to Go ingestion service.
2. Expose metrics from Python workers.
3. Create a small Grafana dashboard.
4. Add basic OpenTelemetry tracing from ingestion → queue → inference.

7.6 Track F – Product & UX Thinking

Goals:

- Make the tool usable and trustworthy to real users.

Topics:

- Designing explanations that non-ML people understand.
- Alert fatigue and notification strategy.
- Feedback UX (simple forms, easy labeling).
- Documentation and onboarding.

Action Plan:

1. Draft example alert messages and anomaly explanations.
2. Create mockups (even on paper) for the dashboard.
3. Define a simple user flow:
    - See alert → understand reason → give feedback.

8. Phased Execution Plan

Phase 1 – Minimal End-to-End (1–2 machines)

- Rust agent → Go ingestion → Redis Streams → Python inference → log anomalies.
- Single-node Postgres/TimescaleDB.
- No dashboard yet; just logs or simple CLI.

Phase 2 – Add Explainability & Dashboard

- Integrate SHAP for reasons.
- Build a basic FastAPI dashboard.
- Allow feedback on anomalies.

Phase 3 – Scalability & Hardening

- Introduce K8s deployments, HPA.
- Split out feature workers & inference workers.
- Add TimescaleDB compression, read replicas.

Phase 4 – Multi-Agent Correlation & Drift

- Implement correlation engine and rules.
- Add drift detection & baseline manager.
- Evaluate on more machines/environments.

Phase 5 – Production Polish

- Alert fatigue controls.
- Strong observability setup.
- Security hardening, permission model, anonymization options.

If you want, I can next:

- Turn this into a more formal "design spec" or "project proposal" style document.
- Or break down **Phase 1** into concrete tasks with checklists and rough timelines.